

TUESDAY, SEPTEMBER 22ND, 2026

OPERATIONS RESEARCH

A Price Instead of a Rule — Lagrangian Relaxation and the Art of Choosing What to Break

Delete the constraint that makes a problem hard, charge for breaking it instead, and the quality of the answer depends entirely on which rule you decided to keep.

2,789 words · 15-minute read

Consider a contract manufacturer with three production lines and six orders to place this week. Every order can go on any line, but lines differ: one is fast and expensive, another is slow and cheap, a third is somewhere between. Each line has a fixed number of hours available. The job is to place all six orders at the lowest total cost without overrunning any line.

The obvious thing to do is put every order on whichever line does it most cheaply. That schedule costs 92,000 dollars, and it is useless, because it piles 25 hours of work onto a line that has 23 available. The obvious repair is to relax the problem: let orders be split into fractions, solve the resulting linear program, and take its value as a floor on what any real schedule can cost. That gives 93,200 dollars. It is a genuine bound, and it is also disappointing: the true cheapest legal schedule costs 104,000, so the linear program is wrong by more than ten per cent. Inspecting its answer shows why. Five orders are placed cleanly on one line each. The sixth is split 0.1 on one line and 0.9 on another — a tenth of an order, which is not a thing that exists.

Order	Line A	Line B	Line C
1	21 / 4 h	12 / 6 h	18 / 4 h
2	19 / 8 h	9 / 18 h	20 / 10 h
3	20 / 17 h	30 / 8 h	23 / 4 h
4	26 / 11 h	28 / 19 h	14 / 20 h
5	22 / 16 h	22 / 8 h	17 / 5 h
6	20 / 5 h	24 / 4 h	20 / 8 h
hours available	28	29	23

There are 729 ways to assign six orders to three lines and only 183 of them respect the hours. Enumeration settles the instance in a millisecond. The interest is not in this instance but in what happens when there are six hundred orders instead of six, at which point enumeration is hopeless and the quality

of the bound is the difference between proving optimality in a minute and grinding for a week. And the route to a better bound turns out to run through a move that looks, at first, like vandalism.

Delete The Rule, Post A Price

Suppose you delete a constraint entirely and charge for violating it. Write the problem as minimizing $\mathbf{c}^\top \mathbf{x}$ over $\mathbf{x} \in \mathbf{X}$ subject also to $\mathbf{Ax} = \mathbf{b}$, where $\mathbf{X}$ collects the structure you intend to keep and $\mathbf{Ax} = \mathbf{b}$ is the family of constraints you have decided is ruining your life. For any vector of multipliers $\mathbf{u}$, define

$$L(\mathbf{u}) = \min_{\mathbf{x} \in \mathbf{X}} \left\{ \mathbf{c}^\top \mathbf{x} + \mathbf{u}^\top (\mathbf{b} - \mathbf{Ax}) \right\}.$$

Two things are true of this object and neither is deep. First, $L(\mathbf{u})$ is a valid lower bound on the original optimum, for every $\mathbf{u}$ whatsoever: any $\mathbf{x}$ that was feasible before is still in $\mathbf{X}$, and for such an $\mathbf{x}$ the added term $\mathbf{u}^\top (\mathbf{b} - \mathbf{Ax})$ is exactly zero, so the minimum over the larger set cannot exceed the true optimum. Second, the minimization is over $\mathbf{X}$ alone, which was chosen precisely because minimizing over it is easy. The hard constraints have become a term in the objective — a price list. Break the rule if you like; you simply pay $\mathbf{u}_i$ for each unit by which you break rule i .

What comes back, though, is not a schedule. At the prices that turn out to maximize our shop's harder bound — 17, 14, 25, 26, 27 and 20 for the six orders — the three lines between them take orders one to four and leave orders five and six undone. The prices are optimal and the answer is illegal, and this is the permanent condition of the method rather than a symptom of stopping too early. A Lagrangian relaxation produces bounds, not plans; an actual schedule has to be built afterwards by a heuristic that repairs whatever the subproblem left broken. Everett, writing in 1963, treated the multipliers themselves as the useful output: a set of prices at which independent activities could be turned loose on a shared resource without any central authority deciding who got what [1].

Nor does the gap generally close. In linear programming the dual optimum meets the primal exactly; here it need not, and for our shop it does not. No price vector whatever lifts the bound above 102 against a true cost of 104. Those two remaining points are a property of the problem, not a failure of the search, and they are the reason the method is a bounding device inside a branch-and-bound tree rather than a solver in its own right.

Since every $\mathbf{u}$ gives a bound, the natural thing is to find the best one. That maximization, $\max_{\mathbf{u}} L(\mathbf{u})$, is the Lagrangian dual, and it has a shape worth being precise about. $\mathbf{X}$ is a finite set of points. For each point, the bracket above is an affine function of $\mathbf{u}$. L is therefore the lower envelope of finitely many straight lines — concave, piecewise linear, and non-smooth at every crossing, whatever the problem underneath. The left panel of the figure shows it for our shop with one constraint priced, line A's 28 hours: 326 lines, one per schedule that respects the other two, with a maximum where two of them cross, at a price of $10/27$ of a thousand dollars an hour and a bound of $2719/27 = 100.70$. The best bound is at a corner, and at a corner there is no derivative to set to zero.

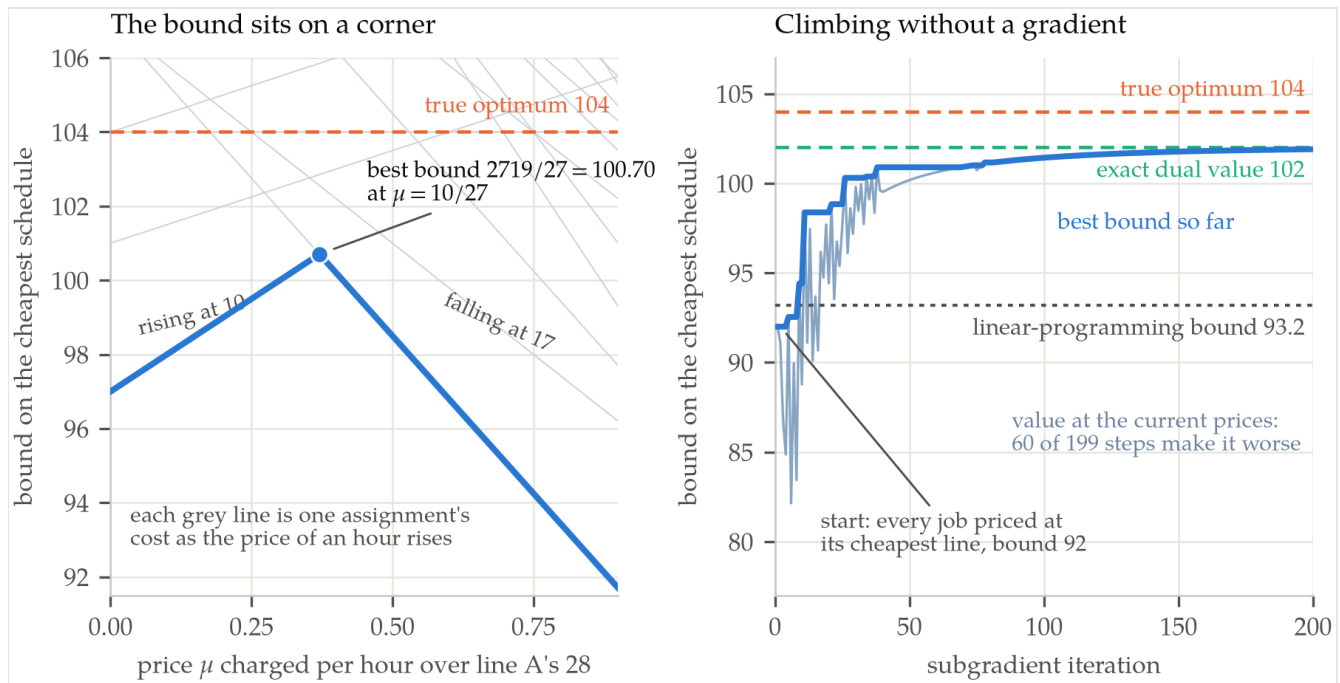


FIGURE The Lagrangian dual of a scheduling problem, and what it takes to climb one. Left: pricing a single capacity constraint produces a concave piecewise-linear function whose maximum is a kink, the envelope rising at 10 to its left and falling at 17 to its right. Right: two hundred steps of subgradient ascent on the dual that prices the six requirement constraints instead.

Climbing A Function With No Slope

The method that took hold is almost embarrassingly simple, and it comes from the same Soviet school — Shor, Polyak, Ermoliev — that was working out how to minimize non-smooth functions in the 1960s [7], [17]. At the current prices, solve the easy subproblem. Look at the answer and measure how badly it breaks the rules you deleted: the violation vector $\mathbf{b} - \mathbf{A}\mathbf{x}$. That vector is a subgradient of $\mathbf{L}$ at $\mathbf{u}$, which is the one piece of luck in the whole construction. It costs nothing extra to compute; solving the subproblem hands it to you. Step in its direction, scaled by some step size, and repeat.

$$\mathbf{u}^{k+1} = \mathbf{u}^k + t_k (\mathbf{b} - \mathbf{A}\mathbf{x}^k), \quad t_k > 0.$$

The interpretation is entirely mercantile. If the subproblem's answer violates a constraint, raise that constraint's price; if it leaves slack, lower it. What makes the method converge is a condition Polyak established in 1969: steps that shrink to zero but whose sum diverges [7]. In practice one uses his other rule, which requires knowing some feasible schedule's cost as a ceiling and scales the step by how far the current bound sits below it.

What the method does not do is improve at every step. The right panel of the figure shows two hundred iterations on our shop's harder dual — the one that prices the six requirement constraints. Sixty of the 199 steps make the bound worse, some of them dramatically, and the value swings across sixteen thousand dollars in the first dozen iterations, from 82,150 up to 98,380. The bound you keep is the running maximum, not the latest value; this is the single most common way to implement the method

wrongly. Held, Wolfe and Crowder’s 1974 paper was titled *Validation of subgradient optimization* for the good reason that the procedure works far better than it looks like it should, and somebody had to demonstrate that experimentally before the field would trust it [5].

TWO RELAXATIONS, ONE PROBLEM

Our shop’s model has two families of constraints: each order must be done exactly once, and no line may be overrun. One relaxation prices the requirements and keeps the capacities. The other prices the capacities and keeps the requirements. Each discards exactly half the model. Is there any reason to expect the two bounds to differ?

Two Ways To Break The Same Problem

Price the requirements, and what remains is one independent knapsack per line: each line, facing a list of orders with reduced costs, picks whichever subset fits in its hours and is most worth taking. Run subgradient ascent on that and the bound climbs to 101.90 after two hundred iterations; solve the dual exactly and it is 102 on the nose. Against a true optimum of 104 and a linear-programming bound of 93.2, that closes 81.5 per cent of the gap.

Price the capacities instead, and what remains is even easier: each order independently goes to whichever line minimizes its cost plus the priced hours it consumes there, with no interaction between orders at all. This is the easier subproblem by a wide margin — and it is worthless. Maximizing that dual returns 93.200000, which is the linear-programming bound to fourteen decimal places.

Bound	Value	Gap to 104
Greedy, cheapest line per order	92.0	infeasible
Linear-programming relaxation	93.2	10.4%
Dual, capacities priced	93.2	10.4%
Dual, requirements priced	102.0	1.9%
True optimum	104.0	—

Two relaxations of one model, each discarding half of it, and one is eight points better than the other. The coincidence in the third row is not a coincidence.

Geoffrion’s Theorem And The Advice It Gives

Arthur Geoffrion settled this in 1974, in the paper that gave the technique its name [4]. The value of the Lagrangian dual is not some mysterious combinatorial quantity. It equals the value of a linear program — specifically,

$$\max_u L(u) = \min \{ c^\top x : x \in \text{conv}(X), Ax = b \}.$$

Dualizing a family of constraints amounts to replacing the retained set by its convex hull and then re-imposing the dualized constraints on top. From which the whole practice follows. If the set you kept was already described by constraints whose linear relaxation has integral vertices — what Geoffrion called the integrality property — then $\text{conv}(X)$ is just the feasible region you wrote down, and the Lagrangian bound is exactly the ordinary linear-programming bound. Not approximately. Exactly.

That is the third row of the table. Keeping only the requirement constraints leaves each order free to pick a line, and the polytope of such choices is a product of simplices, whose vertices are already integral. There was nothing for the convex hull to do, so the bound had nowhere to go. Keeping the capacity constraints leaves three knapsack sets, whose convex hulls are strictly smaller than the boxes containing them, and the difference is the eight points.

The advice this yields runs against every instinct about relaxation. You do not want the easiest subproblem. You want the hardest one you can still solve quickly — a knapsack, a spanning tree, a shortest path, something whose convex hull is genuinely not the thing you would have written down. Marshall Fisher's 1981 survey, the paper that carried the method to practitioners and which *Management Science* reprinted in 2004 among the ten most influential articles of its first fifty years, is largely a catalogue of which constraints to dualize for which problem, and the selection criterion is exactly this [6].

There is even a way to manufacture the difficulty when a model does not supply it. Guignard and Kim's Lagrangian decomposition takes a problem with two constraint families, duplicates the variables, and prices the equation saying the two copies agree — leaving two subproblems, each with one family intact, and a bound at least as strong as either ordinary relaxation [8]. The trick is to dualize a constraint that was not in the model until you put it there.

Held, Karp, And A Bound Within One Per Cent

The canonical demonstration came four years before the theorem. In 1970 and 1971, Michael Held and Richard Karp attacked the symmetric travelling salesman problem by noticing that a tour is a connected subgraph in which every city has degree two, and that if you drop the degree conditions what remains — a 1-tree — is computable by a spanning-tree algorithm in almost no time [2], [3]. So price the degree conditions. Each city gets a number added to the cost of every edge touching it; the minimum 1-tree under the adjusted costs is recomputed; cities with three edges get more expensive and cities with one get cheaper. Part II introduced the ascent scheme that became subgradient optimization [3]. Their branch-and-bound solved every instance put to it, up to sixty-four cities, which in 1970 was a striking claim.

The bound they obtained is, by Geoffrion's theorem, an ordinary linear-programming bound: the minimum over the convex hull of 1-trees subject to the degree constraints. That convex hull is described by the subtour elimination inequalities of Dantzig, Fulkerson and Johnson's 1954 paper [9] — one

inequality for every subset of cities, which is to say exponentially many in the natural edge variables. Held and Karp obtained the value of a linear program nobody could write down, by repeatedly solving minimum spanning tree problems. That is the sentence to keep. Lagrangian relaxation never beats linear programming; it beats the linear program you could afford to state.

How good is the resulting bound? Johnson and McGeoch's experimental work, and the DIMACS challenge instances built around it, report that optimal tour length on randomly generated Euclidean instances averages less than 0.8 per cent above the Held-Karp bound, and that for the real-world instances in TSPLIB the gap is almost always under two per cent [10], [11]. A relaxation that throws away the defining condition of a tour lands within one per cent of the answer.

What Happened Next

The method's most public defeat came in electricity. Deciding which generators to start and stop over a day — unit commitment — is a mixed-integer problem with exactly the structure Lagrangian relaxation was made for: each generator is independent except for the constraint that their outputs must together meet demand. Price demand, and every generator becomes a separate small dynamic program. Every North American system operator ran some version of this for decades. Then branch-and-cut solvers improved enough that solving the integer program directly became faster and gave provably better schedules, and starting with PJM in 2005 the operators switched [14]. MISO's own account of its market redesign, which included the change, put the region's cumulative savings at 2.1 to 3.0 billion dollars over 2007-2010 with 6.1 to 8.1 billion more expected through 2020 [15]. The same happened to the travelling salesman problem, where exact solvers now generate subtour and comb inequalities directly rather than approximating their effect through multipliers.

What survives is everything where the equivalent linear program is still unwriteable. Stochastic programs with thousands of scenarios decompose by pricing the requirement that all scenarios agree on today's decision. Plain subgradient ascent has largely given way to bundle methods, which build a cutting-plane model of the concave dual instead of trusting a single step direction [12], and to Barahona and Anbil's volume algorithm, which recovers an approximate primal solution from the same iterations [13]. And the idea keeps being rediscovered: in 2010 a group at MIT and Columbia introduced dual decomposition to natural language processing, combining a parser and a tagger by pricing the constraint that they agree on the analysis, and showed their bound was the Lagrangian dual of a linear program nobody wanted to solve [16]. Sixty years after a Pentagon analyst wrote it down, the move is still the same one: stop enforcing the rule, and charge for it instead.

Which returns us to that analyst. The 1963 paper that generalized Lagrange multipliers to arbitrary objective functions over arbitrary sets, and which gave the field the resource-allocation framing it still uses, was written at the Weapons Systems Evaluation Division of the Institute for Defense Analyses by Hugh Everett III [1]. Six years earlier, as a doctoral student at Princeton, he had written the thesis proposing that the wavefunction never collapses and every quantum outcome is realized in a separate

branch of reality — the many-worlds interpretation. He then left physics for the Pentagon, headed WSEG's mathematics division from 1956 to 1964, and did not return [18]. The man who argued that no possibility is ever discarded spent his working life computing how to discard them efficiently.

REFERENCES

- [1] H. Everett III, "Generalized Lagrange Multiplier Method for Solving Problems of Optimum Allocation of Resources," *Operations Research*, vol. 11, no. 3, pp. 399-417, 1963.
- [2] M. Held and R. M. Karp, "The Traveling-Salesman Problem and Minimum Spanning Trees," *Operations Research*, vol. 18, no. 6, pp. 1138-1162, 1970.
- [3] M. Held and R. M. Karp, "The Traveling-Salesman Problem and Minimum Spanning Trees: Part II," *Mathematical Programming*, vol. 1, no. 1, pp. 6-25, 1971.
- [4] A. M. Geoffrion, "Lagrangian Relaxation for Integer Programming," *Mathematical Programming Study* 2, pp. 82-114, 1974.
- [5] M. Held, P. Wolfe and H. P. Crowder, "Validation of Subgradient Optimization," *Mathematical Programming*, vol. 6, pp. 62-88, 1974.
- [6] M. L. Fisher, "The Lagrangian Relaxation Method for Solving Integer Programming Problems," *Management Science*, vol. 27, no. 1, pp. 1-18, 1981; reprinted in *Management Science*, vol. 50, no. 12, 2004.
- [7] B. T. Polyak, "Minimization of Unsmooth Functionals," *USSR Computational Mathematics and Mathematical Physics*, vol. 9, no. 3, pp. 14-29, 1969.
- [8] M. Guignard and S. Kim, "Lagrangian Decomposition: A Model Yielding Stronger Lagrange Bounds," *Mathematical Programming*, vol. 39, no. 2, pp. 215-228, 1987.
- [9] G. Dantzig, R. Fulkerson and S. Johnson, "Solution of a Large-Scale Traveling-Salesman Problem," *Operations Research*, vol. 2, no. 4, pp. 393-410, 1954.
- [10] D. S. Johnson and L. A. McGeoch, "The Traveling Salesman Problem: A Case Study in Local Optimization," in *Local Search in Combinatorial Optimization*, E. H. L. Aarts and J. K. Lenstra, Eds. Chichester, U.K.: Wiley, 1997, pp. 215-310.
- [11] D. S. Johnson, L. A. McGeoch and E. E. Rothberg, "Asymptotic Experimental Analysis for the Held-Karp Traveling Salesman Bound," in *Proc. 7th ACM-SIAM Symposium on Discrete Algorithms*, 1996, pp. 341-350.
- [12] C. Lemaréchal, "Lagrangian Relaxation," in *Computational Combinatorial Optimization*, M. Jünger and D. Naddef, Eds., Lecture Notes in Computer Science, vol. 2241. Berlin: Springer, 2001, pp. 112-156.
- [13] F. Barahona and R. Anbil, "The Volume Algorithm: Producing Primal Solutions with a Subgradient Method," *Mathematical Programming*, vol. 87, no. 3, pp. 385-399, 2000.
- [14] B. Knueven, J. Ostrowski and J.-P. Watson, "On Mixed-Integer Programming Formulations for the Unit Commitment Problem," *INFORMS Journal on Computing*, vol. 32, no. 4, pp. 857-876, 2020.
- [15] B. Carlson et al., "MISO Unlocks Billions in Savings Through the Application of Operations Research for Energy and Ancillary Services Markets," *Interfaces*, vol. 42, no. 1, pp. 58-73, 2012.
- [16] A. M. Rush, D. Sontag, M. Collins and T. Jaakkola, "On Dual Decomposition and Linear Programming Relaxations for Natural Language Processing," in *Proc. Conf. Empirical Methods in Natural Language Processing*, 2010, pp. 1-11.
- [17] M. A. Bragin, "Survey on Lagrangian Relaxation for MILP: Importance, Challenges, Historical Review, Recent Advancements, and Opportunities," *Annals of Operations Research*, 2023.
- [18] P. Byrne, *The Many Worlds of Hugh Everett III: Multiple Universes, Mutual Assured Destruction, and the Meltdown of a Nuclear Family*. Oxford, U.K.: Oxford University Press, 2010.

Daily Learning No. 026 · Operations Research · Tuesday, September 22nd, 2026 · Sources are listed above; every load-bearing figure traces to a numbered reference.